



# 도커 심화 실습

볼륨 마운트 · Docker Compose

## 2일차 · 오전

실습 환경: 1일차 실습 VM 계속 사용

## PART 1

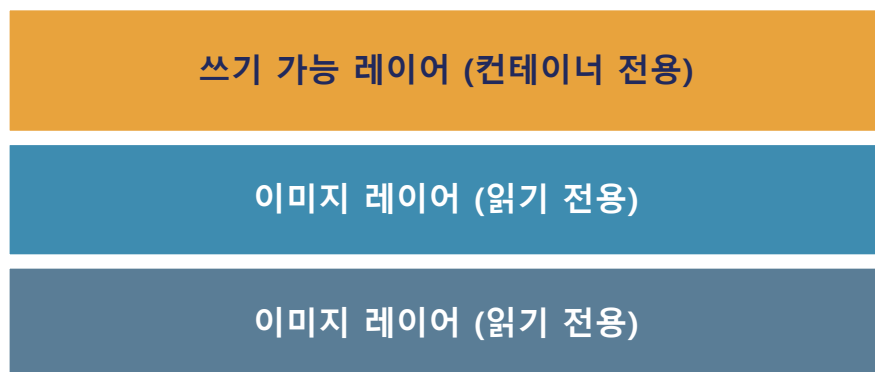
# 볼륨과 데이터 관리

컨테이너가 삭제되어도 데이터를 보존하는  
볼륨 · 바인드 마운트를 실습한다.



# 문제 제기: 컨테이너의 데이터는 휘발된다

쓰기 레이어는 컨테이너와 함께 사라진다



컨테이너 안에서 생성 · 수정한 파일은  
모두 쓰기 레이어에 기록된다



## docker rm 실행 시

쓰기 레이어가 통째로 삭제된다.  
DB 데이터, 업로드 파일, 로그 — 컨테이너에 저장  
한 모든 것이 소실.

**해결 방향** 데이터를 컨테이너 밖(호스트 영역)에 두고 연결한다 — 볼륨(Volume)과 바인드 마운트(Bind Mount)

**원칙:** 컨테이너는 언제든지 갈아끼우는 일회용, 보존할 데이터는 반드시 외부 저장소에.

# 세 가지 마운트 방식 비교

볼륨 · 바인드 마운트 · tmpfs

| 구분        | 볼륨 (Volume)         | 바인드 마운트          | tmpfs (참고)    |
|-----------|---------------------|------------------|---------------|
| 저장 위치     | 도커 관리 영역            | 호스트 임의 경로        | 메모리 (디스크 미사용) |
| 관리 주체     | 도커가 관리              | 사용자가 직접 관리       | 도커가 관리        |
| 주 용도      | DB 등 영속 데이터 (운영 표준) | 소스 코드 공유 (개발 편의) | 민감한 임시 데이터    |
| 재시작 시 데이터 | 유지됨                 | 유지됨              | 소멸됨 (설계된 특성)  |

**오늘 실습 범위** 볼륨과 바인드 마운트를 직접 다루고, tmpfs는 존재와 용도만 소개한다 (세션 규모 밖 심화).

# 볼륨 관리 명령

생성 · 조회 · 상세 · 삭제

## 볼륨 명령

```
# 볼륨 생성
docker volume create mydata

# 목록 조회
docker volume ls

# 상세 정보 (실제 저장 경로 포함)
docker volume inspect mydata
"Mountpoint": "/var/lib/docker/volumes/mydata/_data"

# 삭제 (사용 중이면 실패)
docker volume rm mydata
```



## Mountpoint의 의미

볼륨은 결국 호스트 디스크의 특정 경로에 저장된다.

docker run -v로 명시적으로 만들지 않아도 자동 생성되는 '익명 볼륨'도 존재 — 이름이 없어 관리가 어려우므로 항상 이름을 지정하는 것을 권장.

# 실습 ⑤ 볼륨으로 DB 데이터 보존 (1/2)

볼륨 생성과 MySQL 실행

볼륨 마운트 실행

```
# 1) 볼륨 생성 및 MySQL 실행
docker volume create dbdata
docker run -d --name db \
  -e MYSQL_ROOT_PASSWORD=pass123 \
  -v dbdata:/var/lib/mysql mysql:8.0

# 2) 기동 완료 대기 (수십 초)
docker logs -f db
... ready for connections.

# 3) 테스트 데이터 생성
docker exec -it db mysql -ppass123 \
  -e "CREATE DATABASE edu; SHOW DATABASES;"
```



## 경로 선택의 의미

-v dbdata:/var/lib/mysql — MySQL 이 실제 데이터를 저장하는 디렉터리를 지정해야 한다.

아무 경로나 마운트하면 소용없다 — 애플리케이션마다 데이터 저장 경로가 다르므로 공식 이미지 문서에서 확인한다.

## 실습 ⑤ 볼륨으로 DB 데이터 보존 (2/2)

컨테이너 삭제 후 데이터 생존 검증

### 생존 검증

```
# 4) 컨테이너 완전 삭제 (볼륨은 별개로 존재)
docker rm -f db
docker volume ls    # dbdata는 그대로 남아있음

# 5) 같은 볼륨으로 새 컨테이너 생성
docker run -d --name db \
  -e MYSQL_ROOT_PASSWORD=pass123 \
  -v dbdata:/var/lib/mysql mysql:8.0

# 6) 데이터 생존 확인 → edu 존재
docker exec -it db mysql -ppass123 -e "SHOW DATABASES;"
```



### 검증 논리

컨테이너(실행체)는 4)에서 완전히 폐기되었지만, 데이터는 볼륨에 있으므로 새 컨테이너가 같은 볼륨을 마운트하면 그대로 이어진다.

= 컨테이너 교체와 데이터 보존의 분리

K8s의 PV/PVC가 이 개념의 클러스터 확장판이다.

# 참고: 호스트 경로를 직접 지정하는 방식

명명된 볼륨과 같은 목적, 다른 방법

## 호스트 경로 지정

```
# named volume 대신 호스트 경로를 직접 지정
docker run -d --name database \
  -e MYSQL_ROOT_PASSWORD=test1 \
  -e MYSQL_DATABASE=mydb \
  -v /home/mydb:/var/lib/mysql mysql:5.7

# 호스트에서 파일이 바로 보임 (경로가 명확)
ls /home/mydb
auto.cnf  ib_logfile0  mysql  performance_schema ...
```

|       | named volume    | 호스트 경로              |
|-------|-----------------|---------------------|
| 표기    | dbdata:/var/... | /home/mydb:/var/... |
| 저장 위치 | 도커가 관리          | 내가 지정한 경로           |
| 확인    | inspect 필요      | 바로 ls 가능            |
| 이식성   | 높음              | 경로 의존적              |

**선택 기준** 경로를 직접 들여다보며 디버깅하고 싶다면 호스트 경로 지정을, 이식성과 관리 편의를 원한다면 named volume을 사용한다.



# 실습 다른 컨테이너와 볼륨 공유하기

--volumes-from 옵션

--volumes-from 실습

```
# database 컨테이너가 쓰던 볼륨을 그대로 공유
docker run -it --name exam -h exam \
  --volumes-from database ubuntu

# --- 컨테이너 내부 ---
root@exam:/# ls /var/lib/mysql/
auto.cnf  ib_logfile0  mysql  performance_schema ...
# database와 동일한 데이터가 보인다
```



## 활용 예

백업 전용 컨테이너가 실제 서비스 컨테이너(database)의 볼륨에 접근해 데이터를 백업하는 등, 원본 컨테이너를 건드리지 않고 데이터에 접근할 때 유용하다.

# 실습 ⑥ 바인드 마운트 (1/2)

호스트 디렉터리를 문서 루트로 연결

## 바인드 마운트 설정

```
# 1) 호스트에 웹 문서 디렉터리 생성
mkdir ~/html
echo '<h1>Bind Mount v1</h1>' > ~/html/index.html

# 2) 호스트 디렉터리를 nginx 문서 루트로 마운트
docker run -d --name bindweb -p 8081:80 \
  -v ~/html:/usr/share/nginx/html nginx

# 3) 접속 확인
curl http://localhost:8081
<h1>Bind Mount v1</h1>
```



## 절대경로 주의

바인드 마운트는 반드시 호스트의 절대경로(또는 ~)를 사용한다.

상대경로나 존재하지 않는 이름을 쓰면 도커가 이를 볼륨 이름으로 오인해 새 볼륨을 만들어버리는 대표적인 실수가 발생한다.

## 실습 ⑥ 바인드 마운트 (2/2)

실시간 반영 확인과 읽기 전용 옵션

실시간 반영 · 읽기전용

```
# 4) 호스트에서 파일 수정 → 즉시 반영 확인
echo '<h1>Bind Mount v2</h1>' > ~/html/index.html
curl http://localhost:8081
<h1>Bind Mount v2</h1>

# 5) 읽기 전용으로 마운트 (보안 강화)
docker run -d --name bindweb2 -p 8082:80 \
  -v ~/html:/usr/share/nginx/html:ro nginx
docker exec bindweb2 sh -c \
  "echo test > /usr/share/nginx/html/x.html"
sh: can't create x.html: Read-only file system
```

### 동작 원리

복사가 아니라 동일한 디렉토리를 양쪽에서 바라보는 것(마운트) — 재빌드 없이 수정이 즉시 반영된다

### :ro 옵션

컨테이너에서 해당 경로를 수정 못 하게 강제 — 설정 파일 등 실수로 변경되면 안 되는 대상에 적용

# 볼륨 백업과 복원

임시 컨테이너를 이용한 간단한 방법

## 백업 · 복원 스크립트

```
# 백업 - 볼륨 내용을 tar로 압축해 호스트로 추출
docker run --rm -v dbdata:/data -v $(pwd):/backup \
  busybox tar czf /backup/dbdata_backup.tar.gz -C /data .

# 복원 - 새 볼륨에 압축 파일 풀어넣기
docker volume create dbdata_restored
docker run --rm -v dbdata_restored:/data \
  -v $(pwd):/backup busybox \
  tar xzf /backup/dbdata_backup.tar.gz -C /data
```

**패턴의 핵심** 임시 컨테이너(--rm)가 두 마운트(볼륨 + 호스트 경로)를 동시에 열어 데이터를 옮기는 것 — 별도 백업 도구 없이 도커만으로 가능하다.

## PART 2

# Docker Compose

다중 컨테이너 서비스를 하나의 YAML로 정의하고  
한 명령으로 기동 · 종료한다.



# Docker Compose란

명령 옵션의 나열에서 선언적 구성 파일로

## Before — run 명령의 한계

- 컨테이너마다 긴 run 명령을 순서대로 수동 실행
- 네트워크 · 볼륨 생성도 별도 명령
- 구성이 명령 이력에만 존재 — 재현 · 공유 곤란

## After — Compose

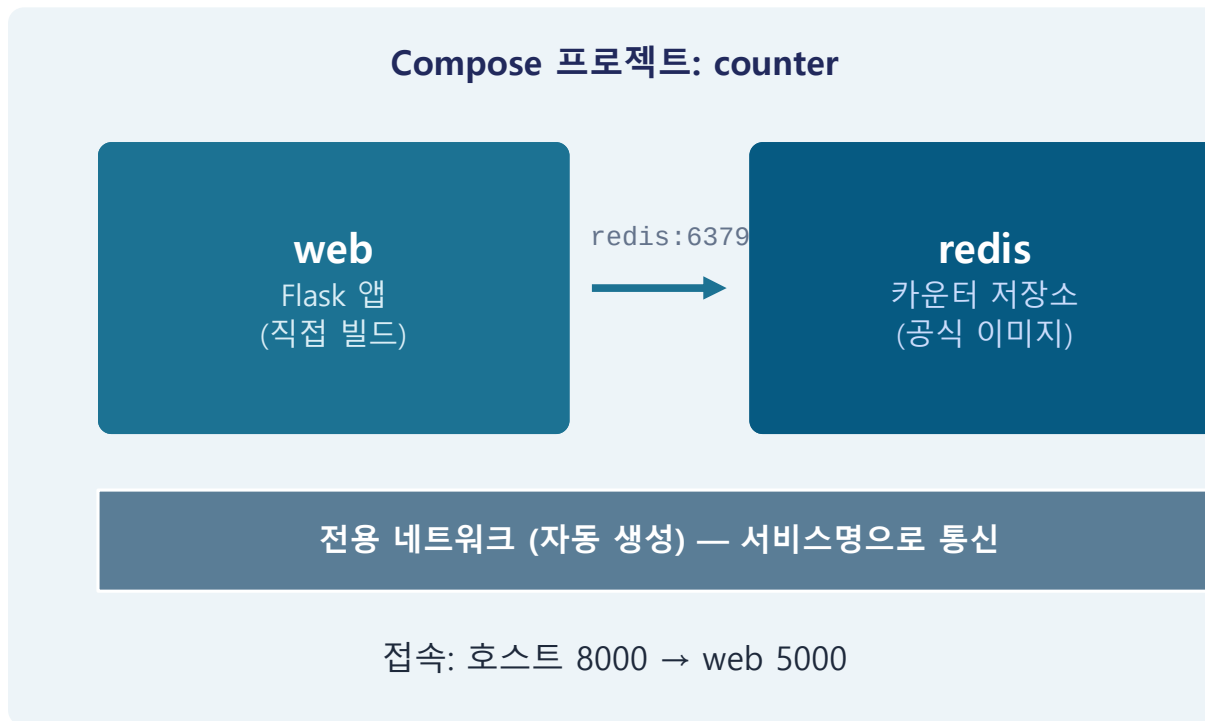
- 전체 구성을 docker-compose.yml 한 파일로 선언
- docker compose up 한 명령으로 전체 기동
- 네트워크 · 볼륨 자동 생성, 파일로 버전 관리

**선언적 방식** "무엇을 어떻게 실행하라"의 절차가 아니라 "이런 상태여야 한다"는 결과를 기술 — K8s와 동일한 철학.

적용 범위: Compose는 단일 호스트용 도구 — 다중 호스트 · 자동 복구는 K8s의 영역 (오후 세션)

# 실습 서비스 구성

웹 애플리케이션 + Redis 방문자 카운터



## 웹 앱 코드 전문

```
# app.py 전체
from flask import Flask
import redis

app = Flask(__name__)
r = redis.Redis(
    host='redis',
    port=6379)

@app.route('/')
def count():
    n = r.incr('hits')
    return f'방문 {n}회'

app.run(host='0.0.0.0',
        port=5000)
```

# 실습 counter 앱 준비

app.py(제공 자료) + requirements.txt(직접 작성)

## requirements.txt 작성

```
# 1) 프로젝트 폴더 준비
mkdir ~/counter && cd ~/counter
# app.py는 제공 자료를 그대로 이 폴더에 저장 (앞 슬라이드의 코드)

# 2) requirements.txt 작성 – 어떤 패키지가 필요한지 명시
cat > requirements.txt << 'EOF'
flask==3.0.*
redis==5.0.*
EOF
```



# 실습 counter 앱 Dockerfile 작성

compose가 build: .로 참조할 이미지 설계도

## Dockerfile 작성

```
# ~/counter 폴더 안에서 계속 진행 (app.py · requirements.txt 준비됨)
cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
EOF

# 확인 — 세 파일이 모두 준비되었는지
ls
```

# docker-compose.yml 작성

서비스 · 포트 · 볼륨 · 의존성 · 헬스체크까지 한 번에

## docker-compose.yml (완성본)

```
# ~/counter/docker-compose.yml
services:
  web:
    build: .
    ports: ["8000:5000"]
    environment:
      - APP_ENV=development
    restart: unless-stopped
    depends_on: [redis]
  redis:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
    volumes: ["redisdata:/data"]
volumes:
```

| 키           | 의미                       |
|-------------|--------------------------|
| services    | 실행할 컨테이너 정의              |
| build/image | 직접 빌드 또는 기존 이미지          |
| ports       | 호스트:컨테이너 포트              |
| environment | docker run -e와 동일        |
| restart     | 재시작 정책(unless-stopped 등) |
| depends_on  | 기동 순서만 보장                |
| healthcheck | "준비 완료" 판단 기준            |
| volumes     | 명명 볼륨 선언 · 마운트           |

**주의** depends\_on은 기동 '순서'만 보장한다 — redis가 완전히 응답 가능한 상태인지는 healthcheck가 판단한다.

# Compose 핵심 명령

프로젝트 단위의 생명주기 관리

| 명령                                        | 기능                      | 비고                       |
|-------------------------------------------|-------------------------|--------------------------|
| <code>docker compose up -d</code>         | 전체 서비스 빌드 + 생성 + 기동     | -d 백그라운드, --build 강제 재빌드 |
| <code>docker compose ps</code>            | 프로젝트 내 서비스 상태 확인        | docker ps와 달리 해당 프로젝트만   |
| <code>docker compose logs -f web</code>   | 특정 서비스 로그 추적            | 서비스명 생략 시 전체             |
| <code>docker compose exec web bash</code> | 실행 중 서비스에 명령 실행         | docker exec의 서비스명 버전     |
| <code>docker compose restart web</code>   | 특정 서비스만 재시작             | 전체 재기동 없이 부분 반영          |
| <code>docker compose down</code>          | 서비스 중지 + 컨테이너 · 네트워크 삭제 | -v 추가 시 볼륨까지 삭제 (주의)     |

**핵심** 개별 컨테이너가 아닌 '프로젝트(디렉터리)' 단위로 묶어 관리한다 — yml이 있는 위치에서 실행.

# 실습 ⑦ Compose 서비스 기동 (1/2)

카운터 서비스 빌드와 실행

## 기동 실습

```
# 1) 프로젝트 준비 (제공 자료 사용)
cd ~/counter && ls
app.py  requirements.txt  Dockerfile  docker-compose.yml

# 2) 전체 기동
docker compose up -d
[+] Running 3/3 network, web, redis Started

# 3) 상태 확인
docker compose ps
NAME                STATUS
counter-web-1       Up
counter-redis-1     Up
```

## 확인 포인트

- ✓ up 한 번으로 네트워크 · 볼륨 · 컨테이너 2개 생성 확인
- ✓ docker compose ps에서 둘 다 Up 상태 확인
- ✓ docker network ls에 프로젝트 전용 네트워크 확인

# 실습 ⑦ Compose 서비스 기동 (2/2)

동작 검증과 부분 재시작

검증 · 재시작

```
# 4) 동작 확인 - 새로고침마다 카운트 증가
curl http://localhost:8000
방문 1회
curl http://localhost:8000
방문 2회 # redis에 누적

# 5) web만 재시작해도 카운트 유지 확인
docker compose restart web
curl http://localhost:8000
방문 3회 # 카운트가 끊기지 않음

# 6) down 후 다시 up - 볼륨 덕분에 유지
docker compose down && docker compose up -d
```



## 핵심 메시지

'상태를 가진 것'(redis + 볼륨)과 '상태 없는 것'(web)의 분리가 이 실습의 핵심.

web은 언제든지 재시작·교체해도 되지만, redis의 데이터(볼륨)는 보존되어야 한다 — K8s의 Deployment/StatefulSet 구분과 연결

# Compose 트러블슈팅

자주 발생하는 문제와 확인 방법

| 증상            | 확인 명령                                            | 흔한 원인                                         |
|---------------|--------------------------------------------------|-----------------------------------------------|
| 서비스가 계속 재시작됨  | <code>docker compose logs &lt;서비스&gt;</code>     | 앱 내부 오류, 의존 서비스 미준비 상태에서 연결 시도                |
| 포트 충돌로 up 실패  | <code>docker compose down</code> (다른 프로젝트 먼저 정리) | 동일 호스트 포트를 다른 프로젝트가 점유 중                      |
| YAML 파싱 오류    | <code>docker compose config</code> 로 파일 검증       | 들여쓰기 오류, 탭 문자 사용                              |
| 코드 수정이 반영 안 됨 | <code>docker compose up -d --build</code>        | <code>build:</code> 는 최초 1회만 — 코드 변경 후 재빌드 필요 |

**config 명령** `docker compose config`는 실제 적용될 설정을 미리 출력해준다 — 문법 오류를 실행 전에 잡을 수 있다.

# 실습 ⑧ 어제 만든 WordPress를 Compose로

--link 없이 같은 서비스를 다시 만들기

compose.yml (db 서비스)

```
# ~/wordpress-compose/compose.yml
services:
  db:
    image: mysql:5.7
    volumes:
      - db_data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: pass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress
```

compose.yml (wordpress 서비스)

```
wordpress:
  depends_on: [ db ]
  image: wordpress
  ports:
    - "8000:80"
  environment:
    WORDPRESS_DB_HOST: db
    WORDPRESS_DB_USER: wordpress
    WORDPRESS_DB_PASSWORD: wordpress
  volumes:
    db_data: {}
```

**어제와 비교** docker run --link wpdb:db 대신, 같은 파일 안에 있으면 서비스명(db)만으로 통신된다 — WORDPRESS\_DB\_HOST: db는 어제와 같은 이름이지만 별명을 수동으로 걸어줄 필요가 없다.

## 실습 ⑧ 실행 및 어제 방식과 비교

명령 여러 줄 vs 파일 하나

실행 · 확인 · 종료

```
# 기동 - 명령 한 줄
docker compose up -d

# 확인
docker compose ps
curl http://localhost:8000
# 워드프레스 초기 설정 화면 HTML 응답

# 종료 (컨테이너 + 네트워크 정리)
docker compose down
```

|       | 어제(run) | 오늘(Compose) |
|-------|---------|-------------|
| 실행 명령 | 2번      | 1번          |
| 연결 방식 | --link  | 서비스명        |
| 재현    | 메모 필요   | 파일 공유       |

**결론** compose.yml 파일 하나만 있으면 누구나 같은 순서, 같은 설정으로 동일한 서비스를 재현할 수 있다 — 이것이 선언적 관리의 힘이다.



# 스케일링, 그리고 Compose의 한계

오후 세션(K8s)으로 넘어가는 이유

## 스케일 시도

```
# web 서비스를 3개로 확장
docker compose up -d --scale web=3
# 단, ports 고정 시 충돌 - 제약 확인
Error: port is already allocated
```

### 확인되는 사실

복제는 되지만 부하 분산기가 없고, 호스트 포트는 하나뿐이다. 트래픽을 여러 복제본에 나눠 줄 방법이 Compose에는 없다.

| 요구 사항       | Compose | K8s        |
|-------------|---------|------------|
| 다중 호스트 배치   | 불가      | 지원         |
| 장애 시 자동 재기동 | 제한적     | 핵심 기능      |
| 부하 분산 (복제본) | 없음      | Service    |
| 무중단 롤링 업데이트 | 없음      | Deployment |
| 자동 확장       | 없음      | HPA        |

**결론** Compose는 단일 호스트 개발 · 소규모 운영까지 — 그 이상은 오케스트레이션(K8s)의 영역이다.

# 종합 실습 미션 (1/2)

볼륨 · Compose 통합

## 미션 1

counter 프로젝트에 nginx 서비스를 추가하고 8080 포트로 노출한다 (image: nginx)

## 미션 2

호스트 ~/html을 nginx 문서 루트로 바인드 마운트하여 정적 페이지를 서비스한다

**진행** 각 미션 완료 시 docker compose ps로 서비스가 3개(web, redis, nginx)로 늘어났는지 확인한다.

# 종합 실습 미션 (2/2)

검증과 심화 도전 과제

**미션 3** docker compose down 후 up — 카운트(볼륨)와 정적 페이지(바인드)가 모두 유지되는지 검증한다

**도전 1** nginx 설정으로 /count 경로를 web:5000으로 프록시한다 (리버스 프록시 — 참고 자료)

**도전 2** redis 서비스에 healthcheck를 추가하고 web의 depends\_on에 condition: service\_healthy를 지정한다

**완료 기준** yml 파일 하나로 서비스 3종이 기동되고, 재기동 후에도 데이터가 유지되면 완료.

# 핵심 정리

2일차 오전 주요 내용 요약

- 1 컨테이너 안에서 만든 데이터는 쓰기 레이어에만 있어 컨테이너를 지우면 함께 사라진다.
- 2 보존할 데이터는 볼륨에, 개발 중 코드 공유는 바인드 마운트에 — 컨테이너는 일회용이다.
- 3 Compose는 다중 컨테이너 구성을 YAML로 선언하고 up/down으로 일괄 관리한다.
- 4 Compose는 서비스명으로 통신할 네트워크를 자동 생성한다 — 어제의 --link가 필요 없어지는 이유다 .
- 5 부하 분산 · 자동 복구 · 다중 호스트는 Compose의 한계 — 쿠버네티스가 필요한 이유다.

2일차 오후

# 다음 세션: 쿠버네티스 기초 실습

아키텍처 이해 · VMware 기반 클러스터 구축 · 기본 리소스



K8s 아키텍처 — 컨트롤 플레인과 워커 노드



kubeadm으로 VM 3대 클러스터 직접 구축



Pod · Deployment · Service 기본 리소스 실습

오후 세션은 사전 구성된 k8s-master · worker1 · worker2 VM 3대를 사용한다.